PYTHON PROGRAMMING ACTIVITY BASED LEARNING(ABL):

Experiment -1. University Scholarship Allocation:

1. Problem Statement:

University Scholarship Allocation: Design a system that evaluates applicants on subject-wise minimum scores, attendance, and ranks eligible students based on the criteria.

2.objectives:

1.To check students' scholarship eligibility.

2.To verify marks, attendance, and family income.

3.To identify eligible students based on the given criteria.

4.To rank eligible students according to their marks.

3. Assumptions:

The minimum aggregate marks required are 80%, with at least 40 marks in each subject. Students must have a minimum attendance of 75%, and their family annual income must not exceed ₹3,00,000. Students with higher aggregate marks receive higher ranks. If no student is eligible, the system displays "No eligible students".

4.Algorithm:

1. Start.
2. Input the number of applicants.
3. Input marks, attendance, and family income.
4. Check the eligibility criteria.
5. Rank eligible students based on aggregate marks.
6. Display the result
7. End.

5. Pseudocode:

```
BEGIN
INPUT marks, attendance, income
   IF marks >= 80 AND attendance >= 75
```

```
        AND income <= 300000
            PRINT "Eligible"
    ELSE
        PRINT "Not Eligible"

    Display result
STOP
```

6.Flowchart: Start → Input applicant details → Check subject marks, aggregate, attendance and income → Check eligibility → Rank eligible students → Display results → End

7.Program code:

```
# University Scholarship Allocation

marks = float(input("Enter aggregate marks (%): "))

subject1 = float(input("Enter marks in Subject 1: "))

subject2 = float(input("Enter marks in Subject 2: "))

attendance = float(input("Enter attendance percentage: "))

income = float(input("Enter annual family income (Rs.): "))

if (marks >= 80 and subject1 >= 40 and subject2 >= 40

    and attendance >= 75 and income <= 300000):

        print("Student is Eligible for Scholarship")

else:

 print("Student is Not Eligible for Scholarship"
```

8.Test Cases:

Test Case	Input	Expected Output
1	Marks: 85%, Attendance: 90%, Income: ₹2,00,000, Subjects: 80, 90	Eligible
2	Marks: 75%, Attendance: 90%, Income: ₹2,00,000, Subjects: 80, 85	Not Eligible
3	Marks: 85%, Attendance: 70%, Income: ₹2,00,000, Subjects: 80, 90	Not Eligible

Test Case	Input	Expected Output
4	Marks: 85%, Attendance: 90%, Income: ₹4,00,000, Subjects: 80, 90	Not Eligible
5	Marks: 85%, Attendance: 90%, Income: ₹2,00,000, Subjects: 35, 90	Not Eligible

9.Sample Output:

Test Case 1: Eligible Student

Enter aggregate marks (%): 85

Enter marks in Subject 1: 80

Enter marks in Subject 2: 90

Enter attendance percentage: 90

Enter annual family income (Rs.): 200000

Student is Eligible for Scholarship

Test Case 2: Marks Below 80%

Enter aggregate marks (%): 75

Enter marks in Subject 1: 80

Enter marks in Subject 2: 85

Enter attendance percentage: 90

Enter annual family income (Rs.): 200000

Student is Not Eligible for Scholarship

Test Case 3: Attendance Below 75%

Enter aggregate marks (%): 85

Enter marks in Subject 1: 80

Enter marks in Subject 2: 90

Enter attendance percentage: 70

Enter annual family income (Rs.): 200000

Student is Not Eligible for Scholarship

Test Case 4: Family Income Above ₹3,00,000

Enter aggregate marks (%): 85

Enter marks in Subject 1: 80

Enter marks in Subject 2: 90

Enter attendance percentage: 90

Enter annual family income (Rs.): 400000

Student is Not Eligible for Scholarship

Test Case 5: Subject Marks Below 40

Enter aggregate marks (%): 85

Enter marks in Subject 1: 35

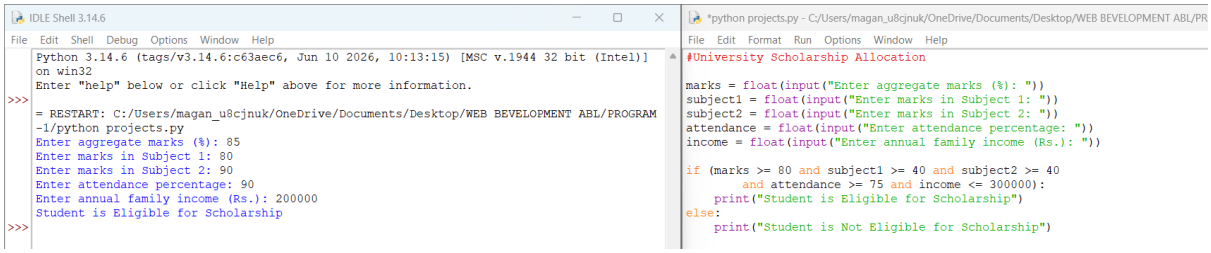
Enter marks in Subject 2: 90

Enter attendance percentage: 90

Enter annual family income (Rs.): 200000

Student is Not Eligible for Scholarship

10:Actual Output:



11.Conclusion:

Thus, the University Scholarship Allocation program was successfully developed using Python. It checks students' marks, attendance, and family income to determine scholarship eligibility. The program helps identify eligible students based on the given criteria.

Experiment -2.Employee Payroll and Deduction Analysis:

1. Problem Statement:

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Objectives:

1. To calculate the gross salary of an employee.
2. To calculate deductions based on salary.
3. To determine the net salary.
4. To generate a formatted salary statement.

3. Assumptions:

The basic pay, allowances, overtime, and incentives are entered by the user. Gross salary is calculated by adding all earnings. Deductions are applied according to the salary amount, and net salary is calculated by subtracting total deductions from gross salary.

4. Algorithm:

1. Start.
2. Input basic pay, allowances, overtime, and incentives.
3. Calculate gross salary.
4. Calculate progressive deductions.
5. Calculate net salary.
6. Display the formatted salary statement.
7. Stop.

5. Pseudocode:

START

INPUT basic pay, allowances, overtime, incentives

gross salary = basic pay + allowances + overtime + incentives

CALCULATE deductions

net salary = gross salary - deductions

DISPLAY salary statement

END

6.Flowchart: Start → Input employee details → Calculate gross salary → Calculate progressive deductions → Calculate net salary → Display salary statement → Stop.

7.Program code:

```
# Employee Payroll and Deduction Analysis

name = input("Enter employee name: ")

basic = float(input("Enter basic pay: "))

allowances = float(input("Enter allowances: "))

overtime = float(input("Enter overtime amount: "))

incentives = float(input("Enter incentives: "))

gross = basic + allowances + overtime + incentives


# Progressive deduction rates

if gross <= 20000:

  deduction = gross * 0.05

elif gross <= 50000:

  deduction = gross * 0.10

else:

  deduction = gross * 0.15

net = gross – deduction


print("\n------ SALARY STATEMENT ------")

print("Employee Name :",name)

print("Basic Pay    :",basic)

print("Allowances   :",allowances)

print("Overtime     :",overtime)
```

```
print("Incentives    :",incentives)
print("Gross Salary  :",gross)
print("Deduction     :",deduction)
print("Net Salary    :",net)
```

8.Test Cases:

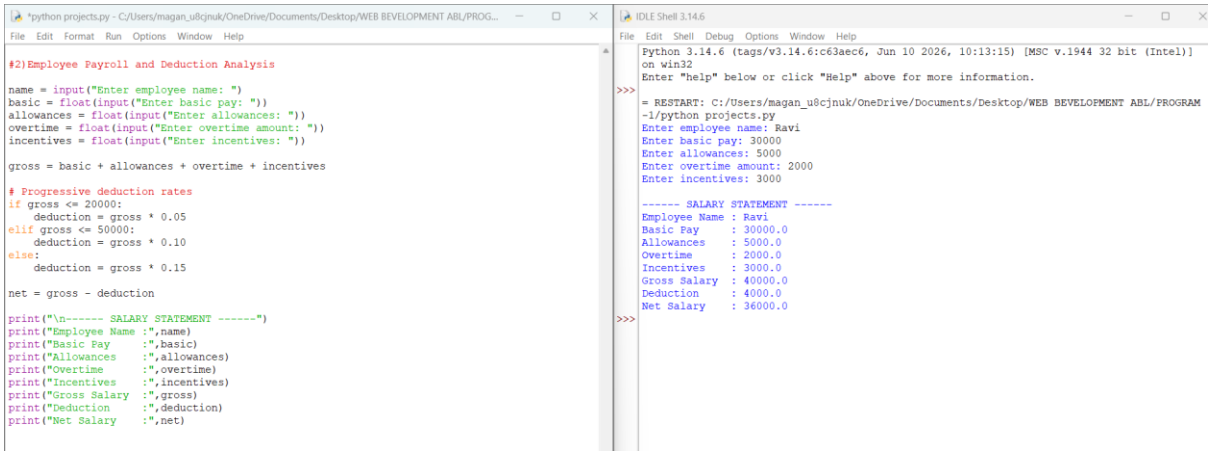
Test Case	Gross Salary	Deduction Rate	Expected Result
1	₹18,000	5%	Net salary: ₹17,100
2	₹35,000	10%	Net salary: ₹31,500
3	₹60,000	15%	Net salary: ₹51,000

9.Sample Output:

```
Enter employee name: Ravi
Enter basic pay: 30000
Enter allowances: 5000
Enter overtime amount: 2000
Enter incentives: 3000

      SALARY STATEMENT
Employee Name : Ravi
Basic Pay     : ₹ 30000.0
Allowances    : ₹ 5000.0
Overtime      : ₹ 2000.0
Incentives    : ₹ 3000.0
Gross Salary  : ₹ 40000.0
Deduction     : ₹ 4000.0
Net Salary    : ₹ 36000.0
```

10:Actual Output:



```
#2)Employee Payroll and Deduction Analysis

name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
allowances = float(input("Enter allowances: "))
overtime = float(input("Enter overtime amount: "))
incentives = float(input("Enter incentives: "))

gross = basic + allowances + overtime + incentives

# Progressive deduction rates
if gross <= 20000:
    deduction = gross * 0.05
elif gross <= 50000:
    deduction = gross * 0.10
else:
    deduction = gross * 0.15

net = gross - deduction

print("\n------ SALARY STATEMENT ------")
print("Employee Name :",name)
print("Basic Pay     :",basic)
print("Allowances    :",allowances)
print("Overtime      :",overtime)
print("Incentives    :",incentives)
print("Gross Salary  :",gross)
print("Deduction     :",deduction)
print("Net Salary    :",net)
```

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:13:15) [MSC v.1944 32 bit (Intel)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/magan_u8cjnuk/OneDrive/Documents/Desktop/WEB BEVELOPMENT ABL/PROGRAM
-1/python projects.py
Enter employee name: Ravi
Enter basic pay: 30000
Enter allowances: 5000
Enter overtime amount: 2000
Enter incentives: 3000

------ SALARY STATEMENT ------
Employee Name : Ravi
Basic Pay     : 30000.0
Allowances    : 5000.0
Overtime      : 2000.0
Incentives    : 3000.0
Gross Salary  : 40000.0
Deduction     : 4000.0
Net Salary    : 36000.0
>>>
```

11. Conclusion

The Employee Payroll and Deduction Analysis program was successfully developed using Python. It calculates gross salary, applies progressive deductions, and displays the employee's net salary in a formatted salary statement.

Experiment 3: Multi-Event Sports Scheduling and Scoring

1. Problem Statement:

To develop a Python program that represents sports event schedules and team scores using lists and tuples, identifies scheduling conflicts, calculates total points, and generates final team rankings.

2. Objectives:

1. To store event schedules using lists and tuples.
2. To identify scheduling conflicts between events.
3. To calculate the total points scored by each team.
4. To rank teams based on their total points.

3. Assumptions:

Each sports event has a name, time, and participating teams. Events scheduled at the same time are considered conflicts. Teams receive points based on their performance. The team with the highest total points receives the first rank.

4. Algorithm:

1. Start.

2. Store sports event schedules using lists and tuples.
3. Check for scheduling conflicts.
4. Input team scores.
5. Calculate total points for each team.
6. Sort teams according to total points.
7. Display conflicts and final rankings.
8. Stop.

5. Pseudocode:

START

STORE event schedules

CHECK scheduling conflicts

STORE team scores

CALCULATE total points

SORT teams by points

DISPLAY conflicts and rankings

END

6. Flowchart in One Line:

Start → Store Event Schedules → Check Scheduling Conflicts → Calculate Team Points → Rank Teams → Display Results → Stop

7. Python Source Code:

```
# Multi-Event Sports Scheduling and Scoring

events = [
  ("Cricket", "10:00 AM"),
  ("Football", "11:00 AM"),
  ("Basketball", "10:00 AM"),
  ("Volleyball", "12:00 PM")
]

print("SPORTS EVENT SCHEDULE")
```

```
for event in events:
    print(event[0], "-", event[1])
print("\nSCHEDULING CONFLICTS")
conflict = False
for i in range(len(events)):
    for j in range(i + 1, len(events)):
        if events[i][1] == events[j][1]:
            print(events[i][0], "and", events[j][0],
                  "have a scheduling conflict.")
            conflict = True
if not conflict:
    print("No scheduling conflicts.")
scores = [("Team A", 30), ("Team B", 45),
          ("Team C", 25), ("Team D", 40)]
scores.sort(key=lambda x: x[1], reverse=True)
print("\nFINAL TEAM RANKINGS")
for rank, team in enumerate(scores, start=1):
    print(rank, team[0], "-", team[1], "points")
```

8. Test Cases:

Test Case	Input Condition	Expected Output
1	Two events at the same time	Scheduling conflict detected
2	All events at different times	No scheduling conflicts
3	Team scores: 30, 45, 25, 40	Teams ranked by descending points

Test Case	Input Condition	Expected Output
4	Team scores are equal	Teams appear in the order retained by the sorting operation

9.Sample Output:

SPORTS EVENT SCHEDULE

Cricket - 10:00 AM

Football - 11:00 AM

Basketball - 10:00 AM

Volleyball - 12:00 PM

SCHEDULING CONFLICTS

Cricket and Basketball have a scheduling conflict.

FINAL TEAM RANKINGS

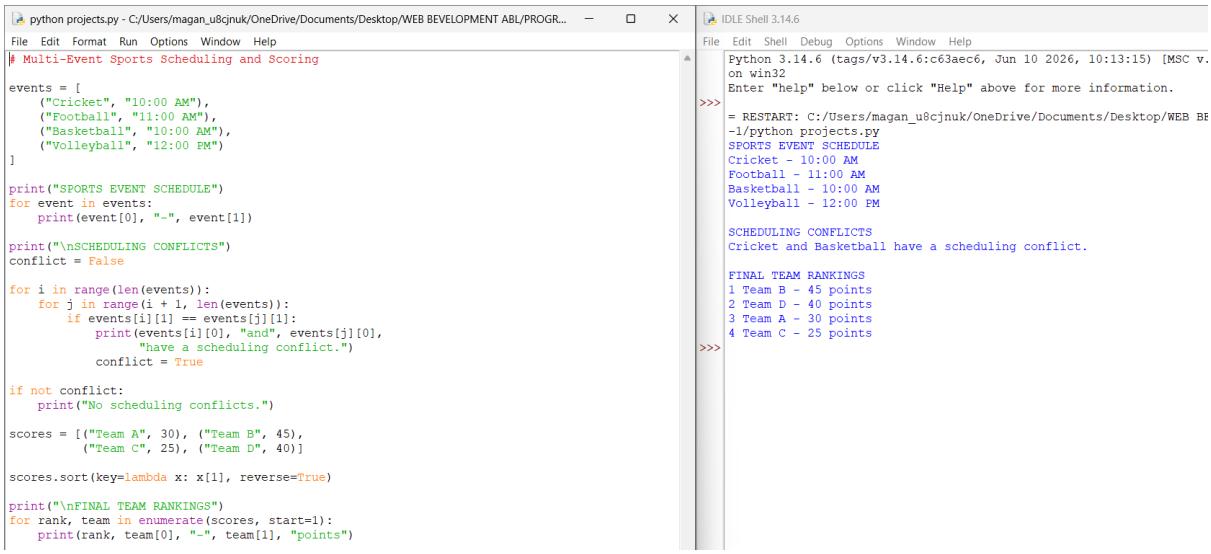
1 Team B - 45 points

2 Team D - 40 points

3 Team A - 30 points

4 Team C - 25 points

10:Actual Output:



11. Conclusion

Thus, the Multi-Event Sports Scheduling and Scoring program was successfully developed using Python. It identifies scheduling conflicts, calculates team rankings based on points, and displays the results.

Experiment 4: Multi-Stage ATM Transaction Audit

1. Problem Statement:

To develop a Python program that processes deposits, withdrawals, and transfers, maintains account balances, detects invalid transactions, and identifies suspicious repeated transaction patterns.

2. Objectives:

1. To process ATM transactions.
2. To maintain account balances.
3. To detect invalid transactions.
4. To identify suspicious repeated patterns.

3. Assumptions:

The account has an initial balance. Deposits increase the balance, while withdrawals and transfers decrease it. Transactions with insufficient funds or invalid amounts are rejected. Repeated withdrawals of the same amount are flagged as suspicious for review.

4. Algorithm:

1. Start.
2. Input the initial account balance.
3. Enter the transaction type and amount.
4. Process deposits, withdrawals, and transfers.
5. Check for invalid transactions.
6. Detect suspicious repeated transactions.
7. Display the final balance and transaction report.
8. Stop.

5. Pseudocode:

START

INPUT initial balance

PROCESS transactions

IF transaction is valid

  UPDATE balance

ELSE

  DISPLAY invalid transaction

CHECK repeated transactions

DISPLAY final balance and report

END

6. Flowchart: Start → Input Initial Balance → Process Transactions → Validate Transactions → Update Balance → Detect Suspicious Patterns → Display Transaction Report → Stop

7. Python Source Code:

Python Source Code – ATM Transaction Audit

```
# Multi-Stage ATM Transaction Audit

balance = float(input("Enter initial balance: "))

transactions = []

withdrawals = []

suspicious = []

n = int(input("Enter number of transactions: "))

for i in range(n):

  print("\n1. Deposit  2. Withdrawal  3. Transfer")

  choice = int(input("Enter transaction type: "))

  amount = float(input("Enter amount: "))

  if amount <= 0:

    print("Invalid transaction amount")

    continue

  if choice == 1:

    balance += amount
```

```
            transactions.append(("Deposit", amount))
            print("Deposit successful")
        elif choice == 2 or choice == 3:
            if amount > balance:
                print("Invalid transaction: Insufficient balance")
                continue
            balance -= amount
            if choice == 2:
                transactions.append(("Withdrawal", amount))
                withdrawals.append(amount)
                print("Withdrawal successful")
            else:
                transactions.append(("Transfer", amount))
                print("Transfer successful")
        else:
            print("Invalid transaction type")
    # Detect repeated withdrawals
    for amount in withdrawals:
        if withdrawals.count(amount) > 1 and amount not in suspicious:
            suspicious.append(amount)
    print("\n----- TRANSACTION REPORT -----")
    for transaction in transactions:
        print(transaction[0], "- Rs.", transaction[1])
    print("Final Balance: Rs.", balance)
    if suspicious:
        print("Suspicious repeated withdrawals:", suspicious)
    else:
        print("No suspicious repeated withdrawals detected")
```

8. Test Cases:

Test Case	Input Condition	Expected Output
1	Deposit ₹1,000	Balance increases
2	Withdraw ₹500 with sufficient funds	Withdrawal successful
3	Withdraw more than the balance	Insufficient balance
4	Enter a negative amount	Invalid transaction amount
5	Repeat withdrawals of the same amount	Suspicious pattern reported

9. Sample Output:

Enter initial balance: 5000

Enter number of transactions: 4

1. Deposit 2. Withdrawal 3. Transfer

Enter transaction type: 1

Enter amount: 1000

Deposit successful

1. Deposit 2. Withdrawal 3. Transfer

Enter transaction type: 2

Enter amount: 500

Withdrawal successful

1. Deposit 2. Withdrawal 3. Transfer

Enter transaction type: 2

Enter amount: 500

Withdrawal successful

1. Deposit 2. Withdrawal 3. Transfer

Enter transaction type: 3

Enter amount: 7000

Invalid transaction: Insufficient balance

----- TRANSACTION REPORT -----

Deposit - Rs. 1000.0

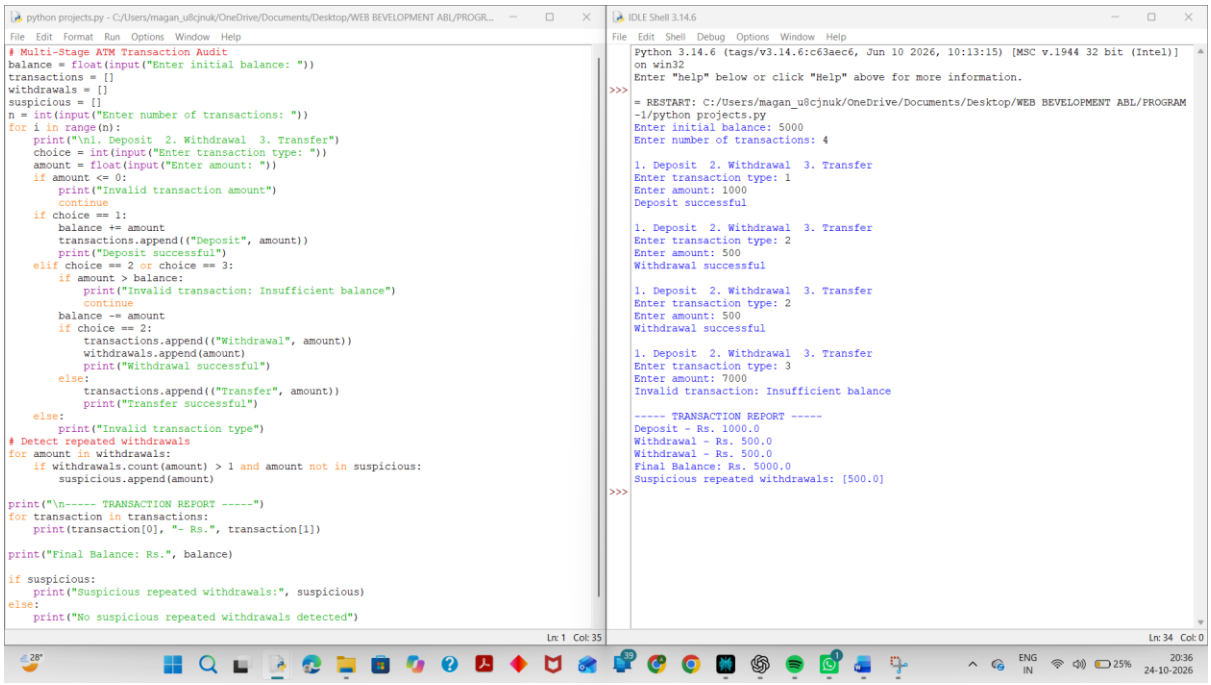
Withdrawal - Rs. 500.0

Withdrawal - Rs. 500.0

Final Balance: Rs. 5000.0

Suspicious repeated withdrawals: [500.0]

10:Actual Output:



11. Conclusion

Thus, the ATM Transaction Audit program was successfully developed using Python. It processes transactions, updates account balances, detects invalid operations, and reports suspicious repeated withdrawals.

Experiment 5: Hospital Resource Allocation and Capacity Monitoring:

1.Problem Statement:

> To develop a Python program that analyses bed occupancy, available staff, and equipment across hospital wards using nested lists, identifies resource shortages, and prioritizes cases based on predefined rules.

2. Objectives:

1. To monitor bed occupancy across hospital wards.
2. To check staff and equipment availability.
3. To identify resource shortages.
4. To prioritize cases based on given rules.

3. Assumptions:

Each ward has a fixed number of beds, available staff, and medical equipment. If occupied beds reach the total capacity, the ward is considered full. If staff or equipment availability falls below the required level, a resource shortage is reported. Cases are prioritized according to the priority rules supplied in the program.

4. Algorithm:

1. Start.
2. Store ward details using nested lists.
3. Check bed occupancy and capacity.
4. Check staff and equipment availability.
5. Identify resource shortages.
6. Assign priority to cases.
7. Display ward status and priority details.
8. Stop.

5. Pseudocode:

START

STORE ward details in nested lists

CHECK bed occupancy

CHECK staff and equipment availability

IDENTIFY resource shortages

PRIORITIZE cases

DISPLAY ward status and shortages

END

6. Flowchart in One Line:

Start → Store Ward Details → Check Bed Occupancy → Check Staff and Equipment → Identify Shortages → Prioritize Cases → Display Report → Stop

7.Python Source Code:

```
# Hospital Resource Allocation and Capacity Monitoring

# Ward: [Name, Total Beds, Occupied Beds, Staff, Equipment]
```

```
wards = [
    ["General", 50, 45, 8, 10],
    ["ICU", 20, 20, 3, 2],
    ["Emergency", 30, 25, 4, 3]
]
print("HOSPITAL RESOURCE REPORT")
for ward in wards:
    name, total, occupied, staff, equipment = ward
    print("\nWard:", name)
    print("Available Beds:", total - occupied)
    if occupied >= total:
        print("Bed Status: Full")
    else:
        print("Bed Status: Available")
    if staff < 5:
        print("Staff Shortage")
    if equipment < 5:
        print("Equipment Shortage")
# Case priority: 1 = High, 2 = Medium, 3 = Low
cases = [
    ("Patient A", 1),
    ("Patient B", 3),
    ("Patient C", 2)
]
cases.sort(key=lambda x: x[1])
print("\nCASE PRIORITY")
for patient, priority in cases:
    print(patient, "- Priority", priority)
```

8. Test Case

Test Case	Condition	Expected Output
1	Occupied beds equal total beds	Ward is full
2	Occupied beds are below capacity	Beds are available
3	Staff count is below 5	Staff shortage reported
4	Equipment count is below 5	Equipment shortage reported
5	Cases have different priority values	Cases sorted by priority

9. Sample Output:

HOSPITAL RESOURCE REPORT

Ward: General

Available Beds: 5

Bed Status: Available

Ward: ICU

Available Beds: 0

Bed Status: Full

Staff Shortage

Equipment Shortage

Ward: Emergency

Available Beds: 5

Bed Status: Available

Staff Shortage

Equipment Shortage

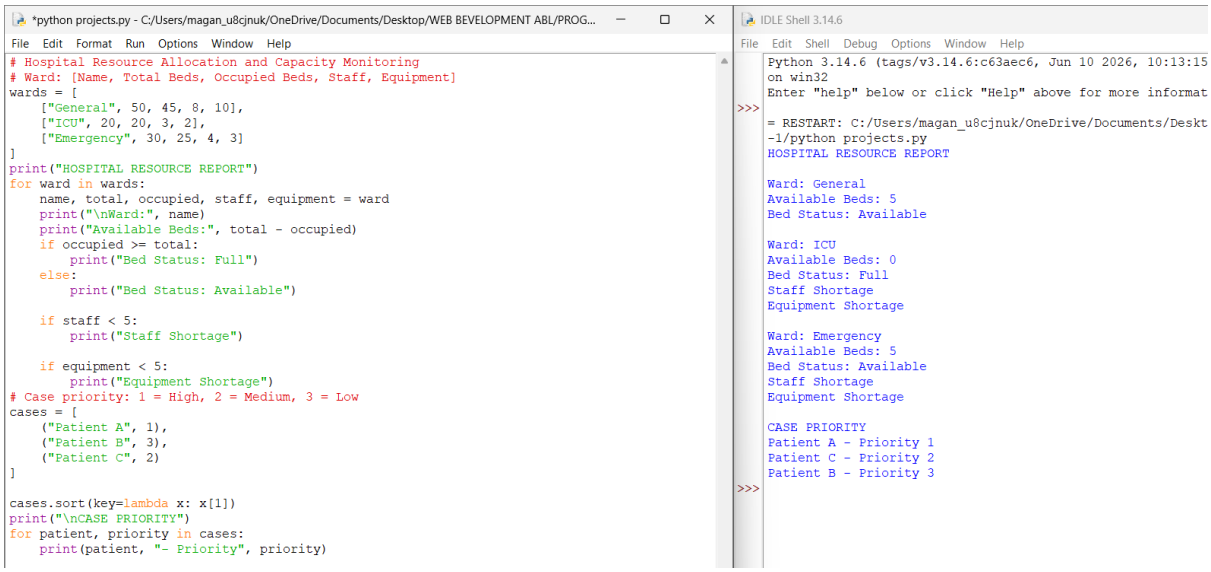
CASE PRIORITY

Patient A - Priority 1

Patient C - Priority 2

Patient B - Priority 3

10:Actual Output:



11. Conclusion

Thus, the Hospital Resource Allocation and Capacity Monitoring program was successfully developed using Python. It monitors ward capacity, identifies staff and equipment shortages, and prioritizes cases according to the defined rules.

Experiment 6: Campus Festival Registration and Financial Audit:

1. Problem Statement:

To develop a Python program that reconciles participant registrations, registration fees, sponsorship contributions, and expenses, detects duplicate registrations, and calculates the final financial balance.

2. Objectives:

1. To maintain participant registration records.
2. To identify duplicate registrations.
3. To calculate registration fees and sponsorship contributions.
4. To calculate total expenses and final financial balance.

3. Assumptions:

Each participant has a unique name. Repeated names are treated as duplicate registrations for this simple example. Registration fees and sponsorship contributions

are added to the total income. Expenses are subtracted from the total income to calculate the final financial balance.

4. Algorithm:

1. Start.
2. Store participant registration details.
3. Identify duplicate registrations.
4. Calculate total registration fees.
5. Calculate total sponsorship contributions.
6. Calculate total expenses.
7. Calculate the final financial balance.
8. Display the audit report.
9. Stop.

5.Pseudocode:

START

INPUT participant details

IDENTIFY duplicate registrations

CALCULATE registration fees

CALCULATE sponsorship contributions

CALCULATE total expenses

BALANCE = TOTAL INCOME - TOTAL EXPENSES

DISPLAY financial report

END

6. Flowchart in One Line:

Start → Store Registrations → Detect Duplicates → Calculate Total Income → Calculate Expenses → Calculate Final Balance → Display Audit Report → Stop.

6.Python Source Code:

```
# Campus Festival Registration and Financial Audit
participants = ["Amrutha", "Ravi", "Sita", "Ravi"]
fee = 500
```

```
sponsorships = [5000, 3000]
expenses = [4000, 2000, 1500]
print("CAMPUS FESTIVAL AUDIT REPORT")
unique_participants = list(set(participants))
duplicates = []
for name in participants:
    if participants.count(name) > 1 and name not in
duplicates:
        duplicates.append(name)
print("\nParticipant List:", participants)
print("Duplicate Registrations:", duplicates)
total_fees = len(unique_participants) * fee
total_sponsorship = sum(sponsorships)
total_expenses = sum(expenses)
total_income = total_fees + total_sponsorship
balance = total_income - total_expenses
print("\nTotal Registration Fees: Rs.", total_fees)
print("Total Sponsorship: Rs.", total_sponsorship)
print("Total Income: Rs.", total_income)
print("Total Expenses: Rs.", total_expenses)
print("Final Financial Balance: Rs.", balance)
```

8. Test Cases:

Test Case	Condition	Expected Output
1	All participant names are unique	No duplicates
2	A participant name is repeated	Duplicate detected
3	Income is greater than expenses	Positive balance
4	Expenses are greater than income	Negative balance
5	Income equals expenses	Zero balance

9. Sample Output:

```
CAMPUS FESTIVAL AUDIT REPORT

participants = ["Amrutha", "Ganavi", "Veda", "Srija"]

Duplicate Registrations: ['Ravi']

Total Registration Fees: Rs. 1500

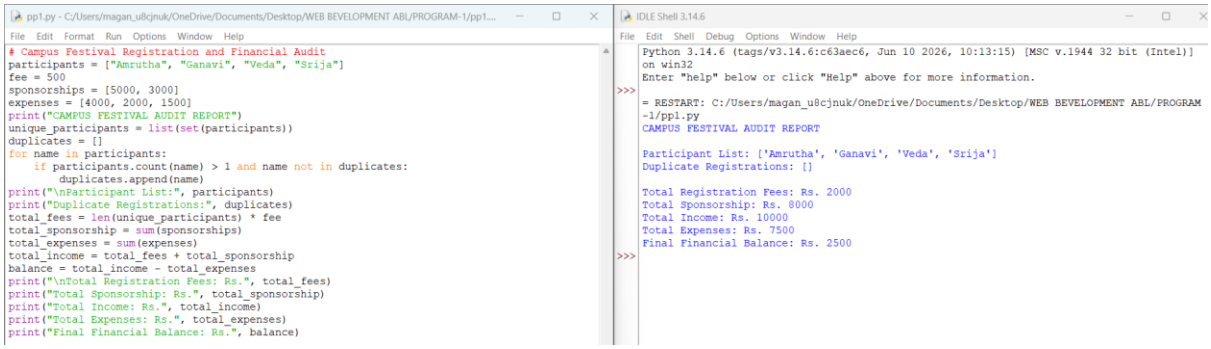
Total Sponsorship: Rs. 8000

Total Income: Rs. 9500

Total Expenses: Rs. 7500

Final Financial Balance: Rs. 2000
```

10:Actual Output:



11. Conclusion:

Thus, the Campus Festival Registration and Financial Audit program was successfully developed using Python. It identifies duplicate registrations, calculates total income and expenses, and displays the final financial balance.

Experiment 7: Student Timetable Conflict Detection:

1. Problem Statement:

To develop a Python program that compares course schedules using strings and tuples, detects overlapping periods, identifies unavailable rooms, and validates a proposed timetable.

2. Objectives:

1. To store course schedules using strings and tuples.

2. To detect overlapping class periods.
3. To identify unavailable classrooms.
4. To validate the proposed timetable.

3. Assumptions:

Each course has a course name, time, and classroom. Two courses scheduled at the same time in the same classroom create a conflict. A classroom marked as unavailable cannot be used for a class.

4. Algorithm:

1. Start.
2. Store course schedules using tuples.
3. Store unavailable classrooms.
4. Compare course timings.
5. Detect overlapping periods.
6. Check classroom availability.
7. Display timetable validation results.
8. Stop.

5.Pseudocode:

START

STORE course schedules

CHECK overlapping periods

CHECK room availability

IF conflicts exist

  DISPLAY conflicts

ELSE

  DISPLAY timetable is valid

Display the timetable validation result

END

6. Flowchart in One Line: Start → Store Course Schedules → Check Time Conflicts → Check Room Availability → Validate Timetable → Display Results → Stop

7.Python Source Code:

```
# Student Timetable Conflict Detection
# Each tuple contains course, time, and room
courses = [
    ("Python", "09:00-10:00", "Room A"),
    ("HTML", "09:00-10:00", "Room B"),
    ("Maths", "10:00-11:00", "Room A"),
    ("English", "11:00-12:00", "Room C")
]
unavailable_rooms = ["Room C"]
print("STUDENT TIMETABLE")
for course in courses:
    print(course)
print("\nCONFLICT REPORT")
```

8. Test Cases

Test Case	Input / Condition	Expected Output
1	Two courses overlap in time	Timetable conflict detected
2	Two courses have different time periods	No time conflict
3	Two courses overlap in the same room	Room conflict detected
4	A course is scheduled in an unavailable room	Unavailable room reported
5	No conflicts and all rooms are available	Timetable is valid

9.Sample Output:

STUDENT TIMETABLE

('Python', '09:00-10:00', 'Room A')

('HTML', '09:00-10:00', 'Room B')

('Maths', '10:00-11:00', 'Room A')

('English', '11:00-12:00', 'Room C')

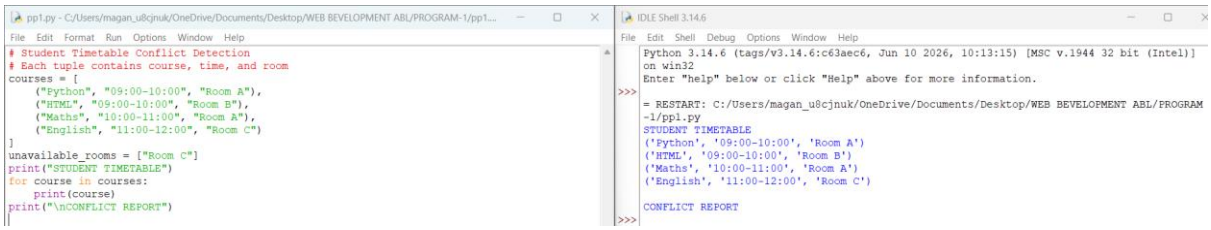
CONFLICT REPORT

Python and HTML have overlapping class timings.

Room C is unavailable for English.

Timetable is INVALID.

10:Actual Output:



11. Conclusion:

Thus, the Student Timetable Conflict Detection program was developed successfully using Python. It helps identify overlapping class timings, detect unavailable classrooms, and validate the proposed timetable. This program helps reduce scheduling conflicts and supports effective timetable management.

Experiment 8: String-Based Examination Answer Evaluation:

1. Problem Statement:

To develop a Python program that compares student answers with an answer key, calculates correct, incorrect, and unanswered responses, applies negative marking, and produces student-wise results.

2. Objectives:

1. To compare student answers with the answer key using strings.
2. To calculate the number of correct, incorrect, and unanswered questions.
3. To apply negative marking for incorrect answers.
4. To calculate the final score of each student.
5. To display student-wise examination results.

3. Assumptions:

1. Each correct answer carries **1 mark**.
2. Each incorrect answer carries a **penalty of 0.25 marks**.
3. Unanswered questions carry **0 marks**.

4. Student answers are compared with the answer key.
5. A blank string represents an unanswered question.

4. Algorithm:

1. Start.
2. Store the answer key.
3. Store student names and their answers.
4. Compare each student's answers with the answer key.
5. Count correct, incorrect, and unanswered responses.
6. Calculate the final score using negative marking.
7. Display the results of each student.
8. Stop.

5. Pseudocode:

```
START
STORE answer key
STORE student names and answers
FOR each student
  SET correct, incorrect, unanswered = 0
  FOR each answer
    IF answer is blank
      INCREASE unanswered
    ELSE IF answer matches the key
      INCREASE correct
    ELSE
      INCREASE incorrect
    END IF
  END FOR
  CALCULATE final score
  DISPLAY student results
```

END FOR

END

6. Flowchart: Start → Store Answer Key and Student Answers → Initialize Counters → Compare Answers → Count Correct, Incorrect and Unanswered → Calculate Final Score with Negative Marking → Display Student-wise Results → Stop.

7. Python Source Code:

```
# String-Based Examination Answer Evaluation

answer_key = ["A", "B", "C", "D", "A"]

students = {

  "Amrutha": ["A", "B", "C", "A", ""],

  "Ravi": ["A", "C", "C", "D", "B"],

  "Sita": ["B", "B", "", "D", "A"]

}

for name, answers in students.items():

  correct = 0

  incorrect = 0

  unanswered = 0

  for i in range(len(answer_key)):

    if answers[i] == "":

      unanswered += 1

    elif answers[i] == answer_key[i]:

      correct += 1

    else:

      incorrect += 1

  score = correct - (incorrect * 0.25)

  print("\nStudent Name:", name)

  print("Correct Answers:", correct)

  print("Incorrect Answers:", incorrect)

  print("Unanswered Questions:", unanswered)
```

```
print("Final Score:", score, "/", len(answer_key))
```

8. Test Cases:

Test Case	Input / Condition	Expected Output
1	All answers are correct	Correct: 5, Score: 5.0
2	All answers are incorrect	Incorrect: 5, Score: -1.25
3	All answers are unanswered	Unanswered: 5, Score: 0.0
4	Some answers are correct, incorrect, and unanswered	Counts and final score are calculated
5	Student gives correct answers with one incorrect answer	Negative marking is applied

9.Sample Output:

Student Name: Amrutha

Correct Answers: 3

Incorrect Answers: 1

Unanswered Questions: 1

Final Score: 2.75 / 5

Student Name: Ravi

Correct Answers: 3

Incorrect Answers: 2

Unanswered Questions: 0

Final Score: 2.5 / 5

Student Name: Sita

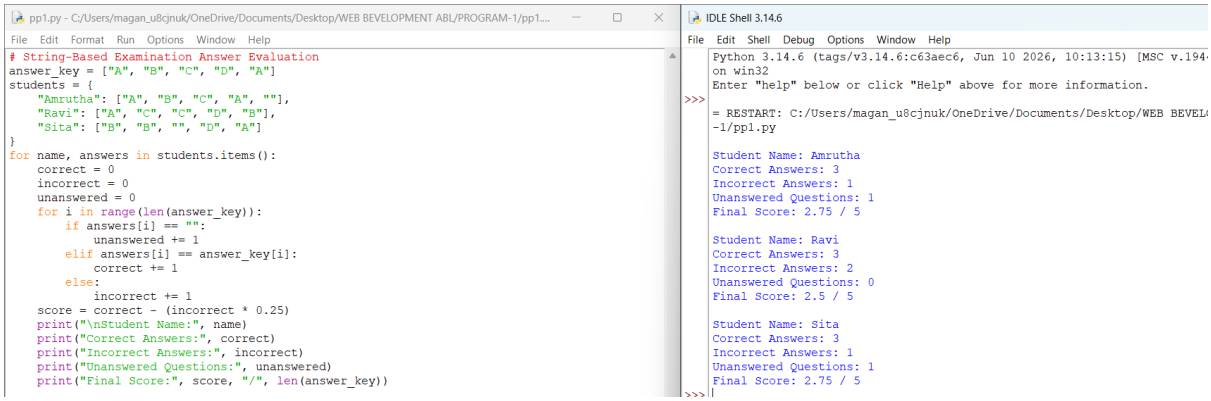
Correct Answers: 4

Incorrect Answers: 0

Unanswered Questions: 1

Final Score: 4.0 / 5

10:Actual Output:



11. Conclusion:

Thus, the String-Based Examination Answer Evaluation program was developed successfully using Python. It compares student answers with the answer key, calculates correct, incorrect, and unanswered responses, applies negative marking, and displays student-wise results.

Experiment -9: University Admission Decision Engine:

1. Problem Statement:

To develop a Python program that evaluates candidates based on subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria, and generates a ranked list of eligible applicants.

2. Objectives:

1. To evaluate candidates based on subject eligibility.
2. To calculate and verify aggregate marks.
3. To apply course-specific admission thresholds.
4. To rank eligible candidates using aggregate marks.
5. To use tie-breaking criteria when candidates have equal marks.

3. Assumptions:

1. Candidates must score at least 60% aggregate marks.
2. Candidates must score at least 50 marks in each required subject.
3. The minimum aggregate for a particular course must be satisfied.

4. Candidates with higher aggregate marks receive higher ranks.
5. If aggregate marks are equal, the candidate with higher subject marks receives the higher rank.

4. Algorithm:

1. Start.
2. Store candidate details and course-specific thresholds.
3. Read each candidate's subject marks and aggregate marks.
4. Check subject eligibility.
5. Check the aggregate and course-specific thresholds.
6. Select eligible candidates.
7. Sort eligible candidates by aggregate marks and apply tie-breaking criteria.
8. Display the ranked list of eligible candidates.
9. Stop.

5. Pseudocode:

```
START
STORE candidate details
STORE course-specific thresholds
FOR each candidate
  CALCULATE aggregate marks
  CHECK subject eligibility
  CHECK aggregate threshold
  CHECK course-specific threshold
  IF all conditions are satisfied
    ADD candidate to eligible list
  END IF
END FOR
SORT eligible candidates by aggregate marks
BREAK ties using subject marks
```

DISPLAY ranked eligible candidates

END

6. Flowchart: Start → Store Candidate Details → Calculate Aggregate Marks → Check Subject Eligibility → Check Aggregate and Course Thresholds → Select Eligible Candidates → Rank Candidates → Apply Tie-Breaking Criteria → Display Ranked List → Stop

7. Python Source Code:

```
# University Admission Decision Engine
candidates = [
  ("Amrutha", 90, 85, 88),
  ("Ravi", 85, 80, 90),
  ("Sita", 90, 85, 92),
  ("Rahul", 70, 65, 68),
  ("Priya", 95, 45, 90)
]
course_threshold = 75
eligible = []
for name, sub1, sub2, sub3 in candidates:
  aggregate = (sub1 + sub2 + sub3) / 3
  if (sub1 >= 50 and sub2 >= 50 and sub3 >= 50
      and aggregate >= 60
      and aggregate >= course_threshold):
    eligible.append(
      (name, aggregate, sub1 + sub2 + sub3)
    )
eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)
print("RANKED LIST OF ELIGIBLE CANDIDATES")
rank = 1
for name, aggregate, total in eligible:
```

```
    print("Rank:", rank,
          "| Name:", name,
          "| Aggregate:", round(aggregate, 2))
    rank += 1
```

8. Test Cases:

Test Case	Condition	Expected Result
1	Candidate satisfies all eligibility criteria	Candidate is eligible
2	Candidate scores below the subject minimum	Candidate is rejected
3	Aggregate is below the course threshold	Candidate is rejected
4	Two candidates have equal aggregate marks	Higher total subject marks determines rank
5	Candidates have different aggregate marks	Higher aggregate receives a higher rank

9. Sample Output:

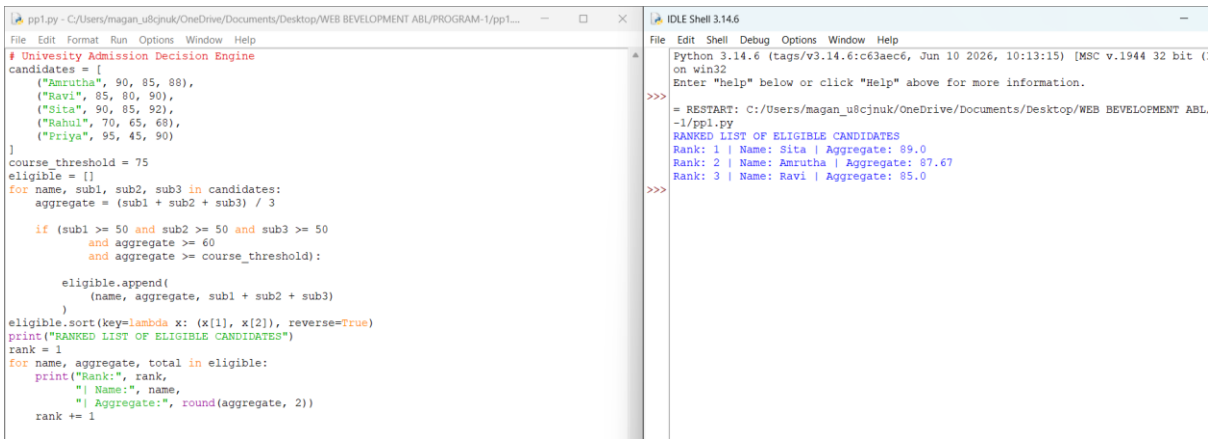
RANKED LIST OF ELIGIBLE CANDIDATES

Rank: 1 | Name: Sita | Aggregate: 89.0

Rank: 2 | Name: Amrutha | Aggregate: 87.67

Rank: 3 | Name: Ravi | Aggregate: 85.0

10:Actual Output:



11. Conclusion:

Thus, the University Admission Decision Engine was developed successfully using Python. It evaluates candidates based on subject eligibility, aggregate marks, and course-specific thresholds. It also ranks eligible candidates and uses tie-breaking criteria to support a systematic and fair admission process.

Experiment-10: Multi-Stage Examination Eligibility System:

1. Problem Statement:

To develop a Python program that determines a student's eligibility for the final examination based on attendance, laboratory completion, internal assessment, and fee clearance, reporting every unmet requirement.

2. Objectives:

1. To check student attendance eligibility.
2. To verify laboratory completion.
3. To evaluate internal assessment marks.
4. To check fee-clearance status.
5. To report all unmet requirements and determine final examination eligibility.

3. Assumptions:

1. Minimum attendance required is 75%.
2. All laboratory experiments must be completed.
3. Minimum internal assessment marks required are 40 out of 100.
4. All examination fees must be cleared.
5. A student is eligible only when all four conditions are satisfied.

4. Algorithm:

1. Start.
2. Read the student's name, attendance, laboratory status, internal marks, and fee status.

3. Check whether attendance is at least 75%.
4. Check whether all laboratory experiments are completed.
5. Check whether internal marks are at least 40.
6. Check whether examination fees are cleared.
7. Report every unmet requirement.
8. If all conditions are satisfied, declare the student eligible.
9. Stop.

5. Pseudocode:

START

INPUT student details

CHECK attendance

CHECK laboratory completion

CHECK internal assessment marks

CHECK fee clearance

IF any requirement is not satisfied

  DISPLAY all unmet requirements

  DISPLAY "Not Eligible"

ELSE

  DISPLAY "Eligible for Final Examination"

Display the result

END

6. Flowchart:

Start → Input Student Details → Check Attendance → Check Laboratory Completion → Check Internal Marks → Check Fee Clearance → Identify Unmet Requirements → Display Eligibility Result → Stop

7. Python Source Code:

```
# Multi-Stage Examination Eligibility System

name = input("Enter student name: ")

attendance = float(input("Enter attendance percentage: "))
```

```
lab_completed = input("Are all labs completed? (yes/no): ").lower()
internal_marks = float(input("Enter internal marks out of 100: "))
fees_cleared = input("Are all fees cleared? (yes/no): ").lower()
requirements = []

if attendance < 75:
    requirements.append("Minimum 75% attendance is required.")
if lab_completed != "yes":
    requirements.append("Complete all laboratory experiments.")
if internal_marks < 40:
    requirements.append("Minimum 40 internal marks are required.")
if fees_cleared != "yes":
    requirements.append("Clear all examination fees.")
print("\nStudent Name:", name)

if requirements:
    print("Status: NOT ELIGIBLE")
    print("Unmet Requirements:")
    for requirement in requirements:
        print("-", requirement)
else:
    print("Status: ELIGIBLE FOR FINAL EXAMINATION")
```

8. Test Cases:

Test Case	Condition	Expected Result
1	All requirements satisfied	Eligible
2	Attendance below 75%	Attendance requirement reported
3	Laboratory work incomplete	Laboratory requirement reported

Test Case	Condition	Expected Result
4	Internal marks below 40	Internal marks requirement reported
5	Fees not cleared	Fee-clearance requirement reported
6	Multiple requirements unmet	All unmet requirements reported

9. Sample Output:

Enter student name: Srija

Enter attendance percentage: 65

Are all labs completed? (yes/no): no

Enter internal marks out of 100: 35

Are all fees cleared? (yes/no): no

Student Name: Srija

Status: NOT ELIGIBLE

Unmet Requirements:

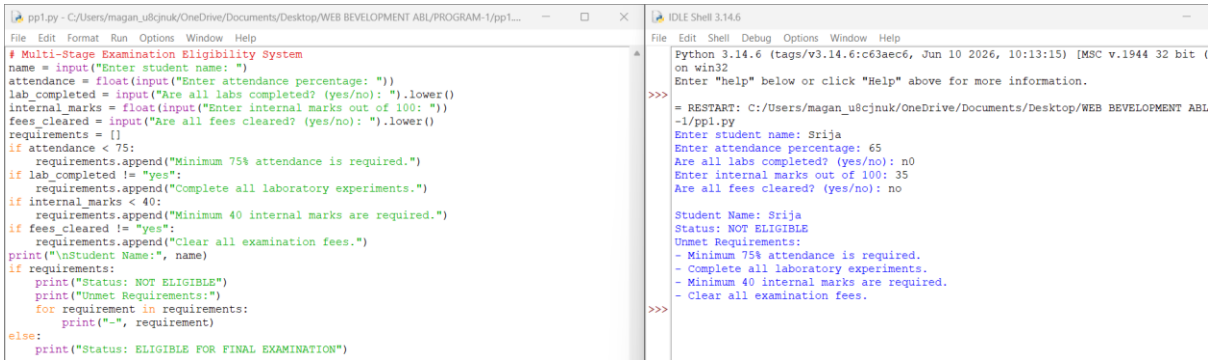
- Minimum 75% attendance is required.

- Complete all laboratory experiments.

- Minimum 40 internal marks are required.

- Clear all examination fees.

10:Actual Output:



11. Conclusion:

Thus, the Multi-Stage Examination Eligibility System was developed successfully using Python. It checks attendance, laboratory completion, internal assessment marks, and

fee clearance. It reports every unmet requirement and determines whether a student is eligible for the final examination.